

Friend function & class

- If we want to access private members inside derived class
- Either we should use member function(getter/setter).
- Or we should declare a facilitator function as a friend function.
- Or we should declare derived class as a friend inside base class.
- Friend function is non-member function of the class, that can access/modify the private members of the class.
- It can be a global function.
- Or member function of another class.
- Friend functions are mostly used in operator overloading.
- If class C1 is declared as friend of class C2, all members of class C1 can access private members of C2.
- Friend classes are mostly used to implement data struct like linked lists.

Operator Overloading

- operator is token in C/C++.
- It is used to generate expression.
- operator is keyword in C++.
- Types of operator:
 1. Unary operator
 2. Binary Operator
 3. Ternary operator
- Unary Operator:
 - If operator require only one operand then it is called unary operator.
 - example : Unary(+,-,*), &, !, ~, ++, --, sizeof, typeid etc.
- Binary Operator:
 - If operator require two operands then it is called binary operator.
 - Example:
 1. Arithmetic operator
 2. Relational operator
 3. Logical operator
 4. Bitwise operator
 5. Assignment operator
- Ternary operator:
 - If operator require three operands then it is called ternary operator.
 - Example:
 - Conditional operator(?:)
- In C/C++, we can use operator with objects of fundamental type directly.(No need to write extra code).

```
int num1 = 10; //Initialization
int num2 = 20; //Initialization
int num3 = num1 + num2; //OK
```

- In C++, also we can not use operator with objects of user defined type directly.
- If we want to use operator with objects of user defined type then we should overload operator.

```
class Point
{
    int x;
    int y;
};
int main( void )
{
    struct Point pt1 = { 10,20};
    struct Point pt2 = { 30,40};
    struct Point pt3;
    pt3 = pt1 + pt2; //Not OK
    //pt3.x = pt1.x + pt2.x;
    //pt3.y = pt1.y + pt2.y;
    return 0;
}
```

- If we want to use operator with objects of user defined type then we should overload operator.
- To overload operator, we should define operator function.
- We can define operator function using 2 ways
 1. Using member function
 2. Using non member function.
- By defining operator function, it is possible to use operator with objects of user defined type. This process of giving extension to the meaning of operator is called operator overloading.
- Using operator overloading we can not define user defined operators rather we can increase capability of existing operators.

Limitations of operator overloading

- We can not overloading following operator using member as well as non member function:
 1. dot/member selection operator(.)
 2. Pointer to member selection operator(.*)
 3. Scope resolution operator(::)
 4. Ternary/conditional operator(? :)
 5. sizeof() operator
 6. typeid() operator
 7. static_cast operator
 8. dynamic_cast operator
 9. const_cast operator
 10. reinterpret_cast operator
- We can not overload following operators using non member function:
 1. Assignment operator(=)
 2. Subscript / Index operator([])

3. Function Call operator[()]
4. Arrow / Dereferencing operator(->)

- Using operator overloading, we can change meaning of operator.
- Using operator overloading, we can not change number of parameters passed to the operator function.

Operator overloading using member function(operator function must be member function)

- If we want to overload, binary operator using member function then operator function should take only one parameter.
- Using operator overloading, we can not change, precedence and associativity of the operator.
- If we want to overload unary operator using member function then operator function should not take any parameter.

```
c3 = c1 + c2; //c3 = c1.operator+(c2);

c4 = c1 + c2 + c3; //c4 = c1.operator+( c2 ).operator+( c3 );
```

Operator overloading using non member function(operator function must be global function)

- If we want to overload binary operator using non member function then operator function should take two parameters.
- If we want to overload unary operator using non member function then operator function should take only one parameters.

```
c3 = c1 + c2; //c3 = operator+(c1,c2);

c4 = c1 + c2 + c3; //c4 = operator+(operator+(c1,c2),c3);

c2 = ++ c1; //c2=operator++( c1 );
```

Overloading Insertion Operator(<<)

-cout is an external object of ostream class which is declared in std namespace.

- ostream class is typedef of basic_ostream class.
- If we want print state of object on console(monitor) then we should use cout object and insertion operator(<<).
- Copy constructor of ostream class is private hence we can not copy of cout object inside our program
- If we want to avoid copy then we should use reference.
- If we want to print state of object(of structure/class) on console then we should overload insertion operator.

```
//ostream out = cout; // NOT OK
ostream &out = cout; //OK
```

1. `cout<<c1; //cout.operator<<( c1 );`
2. `cout<<c1; //operator<<(cout, c1 );`

- According to first statement, to print state of c1 on console, we should define operator<<() function inside ostream class. But ostream class is library defined class hence we should not modify its implementation.
- According to second statement, to print state of c1 on console, we should define operator<<() function globally. Which possible for us. Hence we should overload operator<<() using non member function.

```
class ClassName
{
    friend ostream& operator<<( ostream &cout, ClassName &other );
};

ostream& operator<<( ostream &cout, ClassName &other )
{
    //TODO : print state of object using other
    return cout;
}
```

Overloading Extraction Operator(> >)

- cin stands for character input. It represents keyboard.
- cin is external object of istream class which is declared in std namespace.
- istream class is typedef of basic_istream class.
- If we want to accept data/state of the variable/object from console/keyboard then we should use cin object and extraction operator.
- Copy constructor of istream class is private hence, we can not create copy of cin object in our program.
- To avoid copy, we should use reference.

```
istream in = cin; // NOT OK
istream &in = cin; // OK
```

- If we want to accept state of object (of structure/class) from console(keyboard) then we should overload extraction operator.

1. `cin>>c1; //cin.operator>>( c1 )`
2. `cin>>c1; //operator>>( cin, c1 );`

- According to first statement, to accept state of c1 from console, we should define operator>>() function inside istream class. But istream class is library defined class hence we should not modify its implementation.
- According to second statement, to accept state of c1 from console, we should define operator>>() function globally. Which possible for us. Hence we should overload operator>>() using non member function.

```
class ClassName
{
    friend istream& operator>>( istream &cin, ClassName &other );
};
istream& operator>>( istream &cin, ClassName &other )
{
    //TODO : accept state of object using other
    return cin;
}
```

Index/Subscript Operator Overloading

- If we want to overcome limitations of array then we should encapsulate array inside class and we should perform operations on object by considering it array.
- If we want to consider object as a array then we should overload sub script/index operator.

```
//Array *const this = &a1
int& operator[]( int index )throw( ArrayIndexOutOfBoundsException )
{
    if( index >= 0 && index < SIZE )
        return this->arr[ index ];
    throw ArrayIndexOutOfBoundsException("ArrayIndexOutOfBoundsException");
}
```

//If we use subscript operator with object at RHS of assignment operator then expression must return value from array.

```
Array a1;
cin>>a1; //operator>>( cin, a1 );
cout<<a1; //operator<<( cout, a1 );
int element = a1[ 2 ]; //int element = a1.operator[]( 1 );
```

// If we want to use sub script operator with object at LHS of assignment operator then expression should not return a value rather it should return either address / reference of memory location.

```
Array a1;
cin>>a1; //operator>>( cin, a1 );
a1[ 1 ] = 200; //a1.operator[]( 1 ) = 200;
cout<<a1; //operator<<( cout, a1 );
```

Overloading assignment operator.

- If we initialize newly created object from existing object of same class then copy constructor gets called.
- If we assign, object to the another object then assignment operator function gets called.

```
Complex c1(10,20);  
Complex c2 = c1; //On c2 copy ctor will call  
  
Complex c1(10,20);  
Complex c2;  
c2 = c1; //c2.operator=( c1 )
```

```
class ClassName  
{  
public:  
    ClassName& operator=( const ClassName &other )  
{  
    //TODO : Shallow/Deep Copy  
    return *this;  
}  
};
```

- If we do not define assignment operator function inside class then compiler generates default assignment operator function for the class. By default it creates shallow Copy.
- During assignment, if there is need to create deep copy then we should overload assignment operator function.